The Journal of Systems and Software 219 (2025) 112234

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

DLAP: A Deep Learning Augmented Large Language Model Prompting framework for software vulnerability detection ☆

Yanjing Yang [a], Xin Zhou [a,*], Runfeng Mao [a], Jinwei Xu [a], Lanxin Yang [a], Yu Zhang [a], Haifeng Shen [b], He Zhang [a]

[a] State Key Laboratory of Novel Software Technology, Software Institute, Nanjing University, China
[b] Faculty of Science and Engineering, Southern Cross University, Australia

ARTICLE INFO

Dataset link: https://github.com/Yang-Yanjing/DLAP.git

Keywords:
Vulnerability detection
Large Language Model
Prompting engineering
Framework

ABSTRACT

Software vulnerability detection is generally supported by automated static analysis tools, which have recently been reinforced by deep learning (DL) models. However, despite the superior performance of DL-based approaches over rule-based ones in research, applying DL approaches to software vulnerability detection in practice remains a challenge. This is due to the complex structure of source code, the black-box nature of DL, and the extensive domain knowledge required to understand and validate the black-box results for addressing tasks after detection. Conventional DL models are trained by specific projects and, hence, excel in identifying vulnerabilities in these projects but not in others. These models with poor performance in vulnerability detection would impact the downstream tasks such as location and repair. More importantly, these models do not provide explanations for developers to comprehend detection results. In contrast, Large Language Models (LLMs) with prompting techniques achieve stable performance across projects and provide explanations for results. However, using existing prompting techniques, the detection performance of LLMs is relatively low and cannot be used for real-world vulnerability detections. This paper contributes **DLAP**, a **D**eep **L**earning **A**ugmented LLMs **P**rompting framework that combines the best of both DL models and LLMs to achieve exceptional vulnerability detection performance. Experimental evaluation results confirm that DLAP outperforms state-of-the-art prompting frameworks, including role-based prompts, auxiliary information prompts, chain-of-thought prompts, and in-context learning prompts, as well as fine-turning on multiple metrics.

1. Introduction

Software vulnerability detection is paramount for safeguarding system security and individual privacy. As the cyber environment grows increasingly complex and attack techniques grow quickly, these various threats to software systems have long puzzled software organizations (Lin et al., 2020a; Steenhoek et al., 2023). In particular, vulnerability is one of the critical threats that may result in information leakage, data tampering, and system breaks (Telang and Wattal, 2007). Vulnerability detection aims to identify vulnerabilities, mitigate their impact, and prevent malicious attacks (Gonzalez et al., 2021). Moreover, vulnerability detection helps to enhance software quality, usability, and trustworthiness. Nowadays, vulnerability detection has become a must-have in modern software development.

Many automated static analysis tools (ASATs) have been applied for vulnerability detection (Li et al., 2021; Lin et al., 2020a). However, on the one hand, the outputs of ASATs are difficult to validate as they require developers to master more expertise and experience in vulnerability detection (Nachtigall et al., 2022); on the other hand, the performance of ASATs is poor (*e.g.*, high false positive rates) due to they are based on string pattern matching (Christakis and Bird, 2016; Kang et al., 2022). In recent years, the advancement of deep learning (DL) in natural language processing has inspired researchers to integrate DL models[1] into ASATs. These modern ASATs generally outperform their conventional counterparts in vulnerability detection (Li et al., 2021; Steenhoek et al., 2023). This is because DL models have the advantage of being cost-effective to train and perform well on tasks within specific data distributions. However, DL models that perform well on experimental datasets may suffer from severe performance degradation in real-world projects. This is mainly because of the complexity of the

☆ Editor: Aldeida Aleti.

* Corresponding author.
E-mail address: zhouxin@nju.edu.cn (X. Zhou).

[1] In this paper, 'DL model' refers to the conventional deep learning models other than large language models such as GPT, Copilot, and Llama.

https://doi.org/10.1016/j.jss.2024.112234
Received 1 May 2024; Received in revised form 12 August 2024; Accepted 27 September 2024
Available online 18 October 2024

source code structure and the concealment of vulnerability characteristics (Chakraborty et al., 2022). Using ASATs with DL models to detect vulnerabilities impacts a collection of downstream tasks, including but not limited to vulnerability validation, localization, and repair. Moreover, it is challenging for developers who are responsible for checking vulnerabilities indicated by DL models (Tomas et al., 2019).

In recent years, Large Language Models (LLMs) such as ChatGPT (Brown et al., 2020) and Copilot (Chen et al., 2021) have shown prominent performance in various tasks (Zhang et al., 2023b; Li et al., 2023; Jin et al., 2023). LLMs have the advantage of providing explanations for their answers and maintaining consistent performance on vulnerability detection tasks across different data distributions. However, LLMs have not achieved satisfactory results in vulnerability detection (Purba et al., 2023). Dai et al. (2023) indicated that one of the main reasons is the inappropriate use of LLMs. LLMs are pre-trained by a vast amount of data, but not all of them have positive effects on the downstream tasks such as vulnerability detection (Hsieh et al., 2019; Liu et al., 2023). There are two techniques to address this problem: *fine-tuning* and *prompt engineering*. Fine-tuning is a commonly used technique but requires significant computational resources and time. LLMs with prompts allow users to interact with LLMs iteratively to produce bespoke results (Wei et al., 2022; Dai et al., 2023). However, as detection performance is highly susceptible to prompts, a generic prompting framework would not be able to achieve satisfactory performance (Arakelyan et al., 2023). Prompt engineering makes LLMs adapt to a specific downstream task and generate customized outputs (Wei et al., 2022; Dai et al., 2023). Moreover, prompt engineering can jointly work with fine-tuning, making it a cost-effective and promising technique for vulnerability detection (Shi et al., 2023).

Previous work has utilized LLMs for vulnerability detection using various prompting frameworks (Zhang et al., 2023c; Dai et al., 2023; Purba et al., 2023; Lu et al., 2024). However, existing prompts input limited information to LLMs, providing little help in improving the performance of LLMs in real-world projects. To address this problem, we propose a bespoke prompting framework DLAP.[2] Although DL models cannot perform satisfactorily in multiple projects, they have superior performance in the known project (i.e., a project whose data has already been included in the training set). The core idea of DLAP is using pre-trained DL models for the target project to stimulate adaptive implicit fine-tuning of LLMs. We use the most suitable DL model as a plugin among pre-selected candidate models to augment DLAP and determine whether the source code under inspection contains vulnerabilities. This process is implemented through two state-of-the-art prompts: In-context Learning (ICL) prompt and Chain-of-Thought (COT) prompt. On the one hand, the ICL prompt utilizes locality-sensitive hash (LSH) to sample candidate code fragments that are similar to the input code. Then, a pre-trained DL model is employed to obtain the prediction probabilities of candidate fragments. Combining the candidate code fragments and their corresponding probabilities forms the ICL prompt for the input code. On the other hand, the COT prompt synthesizes the results from static scanning tools and pre-trained DL models as queries. Following these queries, DLAP locates the corresponding COT templates within the detection step template library we constructed based on Common Weakness Enumeration (CWE[3]). Then, DLAP uses these COT templates to generate the customized completed detection COT prompts for input codes. This stimulates LLMs to conduct implicit fine-tuning to achieve better performance in vulnerability detection and provide supplementary information to facilitate the inspection and comprehension of detection results.

We conduct experiments using four large-scale projects with more than 40,000 examples to evaluate DLAP. We first conduct experiments to select the most suitable DL model to form DLAP. We assess performance by integrating various DL models into DLAP and comparing their results to determine the optimal deep learning model. The results show that combining Linevul with an LLM outperformed other DL models by 15% across all evaluation metrics. Then we select the Linevul to generate prompts within DLAP and compare DLAP against the state-of-the-art prompting frameworks including role-based prompts, auxiliary information prompts, chain-of-thoughts prompts, and in-context learning prompts. The results show that DLAP surpasses baselines across all metrics for each project, achieving a 10% higher F1 score and a 20% higher Matthews Correlation Coefficient (MCC). This indicates that DLAP is more effective for vulnerability detection. Finally, we compare our approach with the most prevalent fine-tuning techniques to explore the effectiveness of DLAP versus fine-tuning. The results reveal that DLAP can achieve 90% of the extensive fine-tuning process at a lower cost and even outperform fine-tuning on some metrics. Moreover, the DLAP-driven LLM model generates more explanatory text than fine-tuning, which is important to aid developers in using ASATs for vulnerability detection tasks.

The main contributions of this paper are as follows.

- We propose DLAP, a bespoke LLM prompting framework for vulnerability detection. DLAP combines the advantages of DL models(*i.e.*, easily trainable on a specific task) and LLMs (*i.e.*, high generalization ability between projects and high explainability for detection results) while overcoming their respective shortcomings. Additionally, DLAP has the potential to be adapted for other ASAT tasks.
- We conduct rigorous experiments to demonstrate the effectiveness of selecting appropriate DL models for DLAP and showcase the exceptional vulnerability detection performance over state-of-the-art prompting frameworks.
- We empirically demonstrate the advantages of prompting over fine-tuning for vulnerability detection regarding detection accuracy, cost-effectiveness, and explanations.

The rest of the paper is organized as follows. Section 2 reviews the background and related work. Section 3 delineates the design of DLAP. Section 4 presents the experimental design and parameter setting of DLAP, followed by results and analysis in Section 5. Section 6 discusses DLAP's generation capability and DL model selection. Finally, we present threats to validity in Section 7 and conclude this paper in Section 8.

2. Background and related work

This section describes the related work on vulnerability detection and the background of prompt engineering for LLMs.

2.1. Vulnerability detection

While there is a plethora of work on this topic, we focus on vulnerability detection enhanced by DL and LLMs.

2.1.1. Deep learning for vulnerability detection

Vulnerability detection has received a lot of concerns in recent years. Lin et al. (2020b) proposed a framework incorporating one project and 12 DL models for slice-level vulnerability detection. Zhou et al. (2019) proposed Devign which uses graph representation for input performed better than approaches using tokens of code directly. Li et al. developed a series of DL-based approaches including VulDeePecker, μVulDeePecker, and SySeVR (Li et al., 2021, 2018; Zou et al., 2019) to complete the construction of a DL framework applied to vulnerability detection. Despite achieving advanced results in experimental setups, generalization issues in practical applications still exist. With the advent of networks based on the transformer architecture and language models, researchers have started applying these advanced NLP

[2] Data and materials: https://github.com/Yang-Yanjing/DLAP.git.

[3] https://cwe.mitre.org

techniques to vulnerability detection. Fu and Tantithamthavorn (Fu and Tantithamthavorn, 2022) applied RoBERTa as a pre-training model, fine-tuned on subsequent vulnerability detection tasks, achieving the best experimental performance in both function-level and line-level vulnerability prediction tasks.

Chakraborty et al. (2022) found that the performance of several DL-based approaches dropped by an average of 73% on datasets built from multiple real-world projects, highlighting the need for further research into cross-project vulnerability detection. Steenhoek et al. (2023) conducted an empirical study to demonstrate the variability between runs of a model and the low agreement among DL models' outputs and studied interpretation models to guide future studies.

2.1.2. Large language model for vulnerability detection

The outstanding performance in dialogue, code generation, and machine translation of LLMs has sparked the interest of researchers and practitioners in applying LLMs to software security. Katsadouros et al. (2023) highlighted the potential of LLMs to predict software vulnerabilities, emphasizing their advantage over traditional static methods. Thapa et al. (2022) discovered that transformer-based LLMs outperformed conventional DL-based models. Zhang et al. (2023c) enhanced the effectiveness of ChatGPT in software vulnerability detection through innovative prompt designs and leveraging the model's ability to memorize multi-round dialogue.

However, according to Cheshkov et al. (2023), ChatGPT and GPT-3 in Java code vulnerability detection do not outperform the current tools. In the meantime, Liu et al. (2023) emphasized that ChatGPT cannot replace professional security engineers in vulnerability analysis, indicating that closed-source LLMs are not the end of the story. These findings suggest that the performance of LLMs in the realm of vulnerability detection leaves much to be desired. The potential false positives and illusions generated by LLMs in specific applications (Zhang et al., 2023a) are attributable to the extensive unconstrained training data and many training parameters. Consequently, it is essential to fine-tune an LLM before deploying it for specific tasks. Lu et al. (2024) proposed a GRACE method that processes code structure using CodeT5. It combines semantic with syntactic features to conduct similarity searches and utilizes in-context learning prompts to drive the LLM beyond all baseline DL models on complex real-world datasets.

As indicated above, the performance of LLMs remains unsatisfactory, accompanied by a high false positive rate. In this study, GRACE (Lu et al., 2024) along with other evaluated prompts (Zhang et al., 2023c) will serve as the baselines, facilitating the comparison and evaluation. To achieve better performance in vulnerability detection, we select Sysver (Li et al., 2021), Devign (Zhou et al., 2019), and Linevul (Fu and Tantithamthavorn, 2022) as the augmented components for our framework.

2.2. Prompt engineering for large language models

The increase in the number of parameters in LLMs leads to a rise in the cost of fine-tuning LLMs. Low-cost approaches such as Low-Rank Adaptation of Large Language Models (LoRA) (Hu et al., 2022) and P-tuning (Liu et al., 2022) have significantly reduced the cost of fine-tuning. However, the cost for some applications is still considerable. For example, fine-tuning an LLM with 33B parameters requires two high-precision 40G GPUs (Hu et al., 2022). The parameters of the LLMs that have been reported to achieve excellent performance all exceed one hundred billion, which may cost a lot of computing resources. Research (Brown et al., 2020; Chowdhery et al., 2023; Bai et al., 2022) reveal that LLMs are transformer-based models. Different inputs can cause changes in the attention layers within their architecture, and as such, the construction of high-quality prompts can assist the LLMs in providing satisfactory answers for the specific target tasks. In contrast, inappropriate prompts will impinge on its attention, which may mislead LLMs to produce hallucinations (Zhang et al., 2023a). The COT (Wei et al., 2022) and ICL (Dai et al., 2023) prompting are currently the most effective approaches to prompt engineering. COT prompting is an approach of decomposing a target problem into steps to prompt LLMs to provide the answers. ICL prompts LLMs to deliver correct answers by referring to similar questions. In this paper, DLAP integrates both approaches to provide appropriate prompts to drive LLMs for vulnerability detection.

3. The proposed framework: DLAP

This section describes the design of DLAP, which consists of two prompt techniques.

3.1. Motivation

Vulnerability detection can be formulated as a binary classification problem. Given a vulnerability dataset D that is represented with $D = \{(x_i, y_i)\}_{i=1}^{N}, y = 0$ or 1, where x_i is the source code of a function, and y_i is the ground-truth (0—NO, 1—YES). Detection models are expected to establish their mappings automatically. One of the main processes of detection models is input (source code) representation. Specifically, the source code can be represented as semantic tokens, abstract parsing trees (ASTs), data/control flow graphs (DFGs/CFGs), or other formats. In vulnerability detection, integrating various auxiliary information sources is more effective than relying solely on a single format (Lin et al., 2019). Compared to traditional DL models that require specific designs for different code representations, the unique prompt mechanism of LLMs and their extensive training data allow various forms of code representation to be input as prompts dynamically. The mechanism makes it a more promising detection technique for vulnerability detection. When levering LLMs for vulnerability detection, it is fundamental to make LLMs understand domain and task knowledge, as LLMs are trained by general corpora. It can be expected that using LLMs for vulnerability detection directly would achieve unsatisfactory results. Therefore, LLMs require an augmenting method that improves their vulnerability detection performance without altering their ability to accept diverse prompt inputs. According to recent research, fine-tuning or prompt engineering can address both tasks.

Fine-tuning is an intuitive technique to make the parameters $\mathcal{W}$ of LLMs adapt to downstream tasks (*i.e.*, vulnerability detection) to achieve better results, which can be described as Eq. (1).

$$\mathcal{W} = \text{argmin}\mathcal{L}\left(Y - \mathcal{M}_{\mathcal{W}}(\mathcal{P}(X))\right) \tag{1}$$

where Y is the ground truth (label) for function level code and $\mathcal{L}$ is the loss function. By fine-tuning the weight parameters $\mathcal{W}$ of LLMs, the predictive probabilities are expected to be closer to the ground truth. However, fine-tuning is cost-intensive (Hu et al., 2022; White et al., 2023). For example, LoRA which is one of the most efficient LLMs requires approximately 40G graphics card memory and a lot of time for fine-tuning an LLM with only 13B parameters.

Prompt engineering is a new technique to augment LLMs. LLMs can incorporate various inputs and generate their answers; therefore, they can be prompted (Brown et al., 2020; Chowdhery et al., 2023; Bai et al., 2022). Technically, we use $\mathcal{D}_s^T$ and $\mathcal{D}_o^E$ to describe pretraining sets and testing sets, respectively. When using prompt engineering ($\mathcal{P}(.)$) for vulnerability detection, LLMs accept a set of $\mathcal{P}(X)$ as inputs and output the estimated probabilities of them, where X means a collection of examples from $\mathcal{D}_o^E$. One cost-effective prompting template $\mathcal{P}$ for vulnerability detection can be described as Eq. (2).

$$\mathcal{P} = \text{argmin}\mathcal{L}\left(Y - \mathcal{M}_{\mathcal{W}}(\mathcal{P}(X))\right) \tag{2}$$

According to Liu et al. (2022), Eq. (2) can achieve the same effects as Eq. (1). That is, both prompt engineering and fine-tuning can make predictive probabilities close to ground truths. In the following subsections, we elaborate on DLAP which leverages prompt engineering for vulnerability detection.

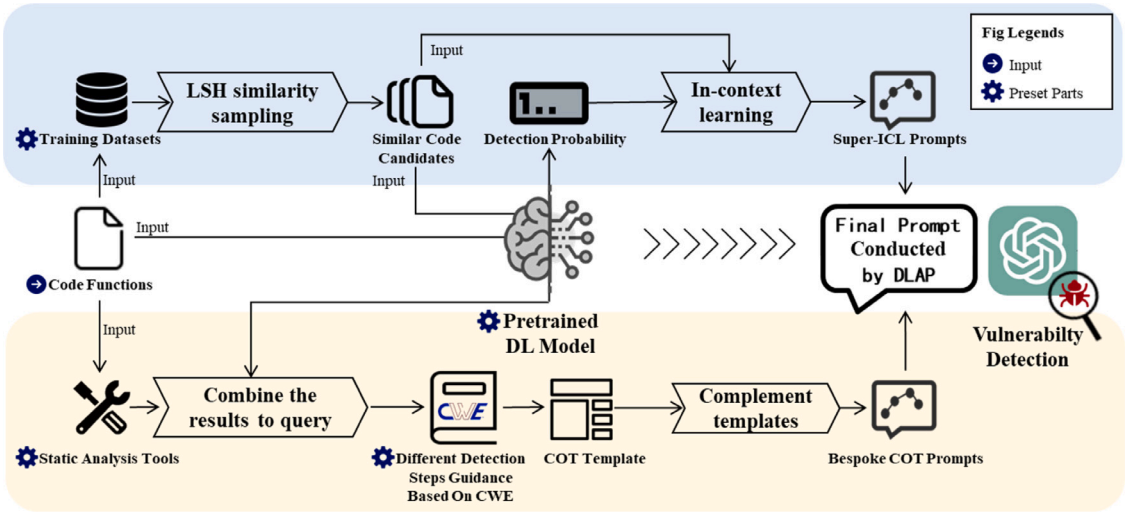

Fig. 1. An overview of the proposed DLAP.

3.2. Framework overview

DLAP leverages attention mechanisms within LLMs, incorporating selectively trained DL models as enhancements. This approach, known as In-Context Learning (ICL), acts to subtly refine LLMs, making them more adept at specific projects. Moreover, DLAP uses chain-of-thoughts (COT) to enable LLMs to discard incorrect generative paths effectively. Consequently, DLAP enhances LLMs' capabilities in detection tasks, ensuring robust performance without incurring significant costs. As a form of prompt engineering, DLAP allows LLMs to accept multiple code representations. ICL can stimulate the attention layer of an LLM to adapt to the downstream detection task, which is defined by Dai et al. (2023) as implicit fine-tuning. As with general fine-tuning, implicit fine-tuning can also drive LLMs to adapt to downstream tasks and achieve better performance. Well-designed prompts stimulate LLMs to perform better in downstream detection tasks. The idea behind the proposed DLAP framework is that it uses DL models to augment LLMs by constructing appropriate prompts to stimulate implicit fine-tuning of the LLMs for them to adapt to vulnerability detection tasks. In this way, it can reduce performance degradation caused by hallucination and data distribution differences.

As shown in Fig. 1, DLAP is composed of two main parts, including (1) Part I: construction of ICL prompts augmented by DL models and (2) Part II: generation of the bespoke COT prompts to augment LLMs. In Part I, we employ DL models to generate detection probabilities for input codes and select candidate codes based on similarity. The combination of candidate codes and their corresponding similarities forms the ICL prompt for detection. In Part II, we combine the results of DL models and static tools to query pre-defined templates in a preset COT template library as key–value pairs. Based on the characteristics of each input sample, we complete the chain of thought, generating COT prompts for detection. These two parts will be introduced in Sections 3.3 and 3.4 respectively. In Section 3.5, we show an example of synergizing the two prompts to generate the prompts of the final DLAP. Using DLAP for vulnerability detection requires two rounds of interaction with LLMs. In the first round, DLAP, combined with the completion capabilities of LLMs, generates bespoke prompts for specific samples. In the second round, these prompts drive the LLM to detect vulnerabilities in the source code and provide explanations, aiding users in understanding the detection results.

3.3. In-context learning prompts construction

According to the earlier point, LLMs encapsulate vast knowledge through their expansive weight structures. We ensured that the LLM adjusts its attention for specific tasks and performs well through a two-step model selection process. In the first step, we selected models that could be integrated into DLAP. Initially, we select pre-trained models designed for binary classification tasks in vulnerability detection. Then, we analyze the data distribution of the target to be detected, including the programming language and preprocessing techniques used. Finally, we train and construct DL models suitable for the specific detection project from the training set. We also design integration interfaces for these models. The input includes source code and the representations required by the pre-trained DL model. The output consists of the results generated by the DL model and the confidence levels calculated based on the model parameters for different outcomes. In the second step, to evaluate DLAP's performance, we select the best DL model from these models selected by the first step based on their performance (cf. Section 5.1).

For new projects, performance degradation might occur if the tested code's semantic and syntactic characteristics significantly deviate from the training set of the pre-trained DL model (Chakraborty et al., 2022; Lin et al., 2018). In such cases, constructing new adaptive datasets and retraining the pre-trained DL model can help mitigate the impact of performance degradation. Then, to create the appropriate in-context, the most similar code candidates are found in the training set using Locality-sensitive hashing (LSH), an efficient similarity search algorithm in the Retrieval Augmented Generation (RAG) technique. Although the LSH similarity calculation algorithm can only concern the similarity of code segments, considering that as a prompting framework, we cannot spend too much time generating prompts formation, it is necessary to efficiently sample multiple codes as a similar code candidate set.

Following Dai et al. (2023), we reversely use the dual form of the attention of transformer derived by them. Therefore, the adaptive implicit fine-tuning on attention layer $\widetilde{\mathcal{A}}$ of LLMs stimulated by DLAP for specific projects can be written as Eq. (3). Please refer to Appendix in the appendix for details.

$$\widetilde{\mathcal{A}}(\mathbf{q}) = (W_{\text{init}} + \Delta W_{ICL}(x))\mathbf{q} \tag{3}$$

We train the DL model $\mathcal{M}$ with training data information `info` which contains the relationship between the project data and the label.

```
[
  { "Input": (code)"static int sb1054_get_regi
  ster(struct sb_uart_port *port, int page, in
  t reg){int ret = 0;unsigned int lcr = 0;unsi
  gned int mcr = 0;...",
  "Detect": "Vulnerability",
  "Probability": 0.003524},

  { "Input": (code)"static int tcp_v4_md5_add_
  func( struct sock *sk, struct sock *addr_sku
  8 *newkey, u8 newkeylen){ return tcp,...",
  "Detect": "Vulnerability",
  "Probability": 0.292371},

  ......

  { "Input": (code)"static void perf_syscall_e
  xit(void * ignore, struct pt_regs *regs, lon
  g ret){ struct syscall_metadata *sys_data; s
  truct syscall_trace_exit *rec…",
  "Detect": "Vulnerability",
  "Probability": 0.993587}
]
```

Fig. 2. An example of ICL prompts of DLAP.

Then the DL model generates a detection probability $\text{Probs}_{\mathcal{P}}$ for a detection object x as shown in Eq. (4).

$$\text{Probs}_{ICL}(Obj_{\text{info}}) = \mathcal{M}(Obj_{\text{info}})(x) \tag{4}$$

The probabilities output by the DL model represent characteristics of input codes. DLAP uses the probabilities to construct ICL prompts to augment LLMs. Then, we obtain the relaxed attention representation $\widetilde{\mathcal{A}}$ of the LLM in Eq. (5).

$$\widetilde{\mathcal{A}}(\mathbf{q}) = \left(W_{\text{init}} + \Delta W(\text{func}(\text{Probs}_{\mathcal{P}}(Obj_{\text{info}})))\right)\mathbf{q} \tag{5}$$

Eq. (5) indicates that the relaxed attention of the LLM is related to probabilities output by the selection of a DL model and the probabilities are related to the training project data. This results in an implicit fine-tuning for the LLM to adapt to specific project information (Dai et al., 2023). We further explain this adaptive process with more details in the result analysis section through a comparative analysis experiment.

Compared to conventional fine-tuning methods, DLAP does not require excessive resource consumption to update the parameters of LLMs. The ICL prompts update the output of the attention layer in the LLMs. As the example shown in Fig. 2, the ICL prompts of DLAP stimulate implicit fine-tuning of the LLMs toward adapting to the characteristics of the projects to be detected.

By adding the results generated by the DL model to the prompts of LLM, DLAP can include more in-context information. The DL model trained to form weights of networks contains the complex relationship between the predicted probability and the input code text. Therefore the DL model's output contains the characteristics of training sets. ICL built in this way contains more prompt information than normal ICL, which stimulates LLMs to perform better in downstream tasks.

As shown in the upper part of Fig. 1, after conducting the detection probability of the candidate codes through the DL model, we set the code candidate set and the corresponding probability as the question and answer combination. These combinations (an example shown on Fig. 2) are constructed to be the in-context learning (ICL) prompt together with part II in Section 3.4 to form the final DLAP augmented prompts.

3.4. Chain-of-thought prompts generation

The second part of DLAP is to generate specific prompts for each tested sample. It is divided into the following stages. Firstly, because the characteristics and detection steps of vulnerabilities vary, we need to pre-set different detection templates into a COT library. According to the existing peer-reviewed vulnerability taxonomies (*i.e.*, Wei et al., 2021; Li et al., 2017) and reliable grey literature (*i.e.*, Tsipenyuk et al., 2005), we construct a hierarchical detection COT library that has six major categories as follows.

- **SFE** (Security Features Errors): Errors induced by imperfect security features
- **LOG** (Logistics Errors): Errors induced by program execution
- **MEM** (Memory Errors): Errors related to memory resources
- **NUM** (Numeric Errors): Errors induced by numerical computations
- **IDN** (Improper Data Neutralization): Errors induced by non-standardization (verification, restriction) of exchanged data
- **UNT** (Unknown Taxonomy Errors): Unknown errors

Subsequently, according to the parent–child relationship described in the CWE research concept,[4] some categories above are refined (a total of 45 subcategories). In addition, by referring to the relevant research (Liu et al., 2023; Zhang et al., 2023c; Ozturk et al., 2023) on step-by-step solutions to vulnerability detection through LLMs driven by the COT, we establish a general paradigm for the generation of the COT as follows.

- **Semantics**: Comprehending the function of the code.
- **Logic**: Analyzing the structure of the code.
- **Internal risks**: Identifying components that may introduce vulnerabilities.
- **External risks**: Inspecting for unsafe functions that could potentially lead to vulnerabilities.
- **Generating the COT**: Integrating the information acquired above and generating a COT to inquire about whether there are potential vulnerabilities step by step.

The specific COT refines the generation paradigms for corresponding COT guidance for different categories. Each subcategory is associated with a specific detection COT template guidance. DLAP selects two open-source functional-level vulnerability detection static tools, *i.e.*, Flawfinder[5] and Cppcheck,[6] to generate static scanning results. It parses the result text of the static tools and maps them to the corresponding categories in the taxonomy tree. Then the results are scored and recorded for each tool. The highest-scoring K categories are taken out and added to the query *key*. DLAP selects the same DL model in Section 3.3 because of their better performance according to the studies (Zhou et al., 2019; Fu and Tantithamthavorn, 2022; Li et al., 2021). DLAP combines the detection results of the DL model and the scanning results of the static tool to become the *key* of a query. Using this *key*, DLAP obtains customized COT generation guidance templates from the COT library for the test codes. The *key* is formed as a dictionary that contains the static tool output class and the result of the DL model judgment, such as the following Fig. 3.

For instance, if the *key* is the null pointer dependency that falls under the IDN category, then DLAP will get the refined COT guidance from the taxonomy tree as shown in Fig. 4. The library of COT guidance templates is publicly available on GitHub.[7]

Through the *key* generation process described earlier, the COT guidance and the results of the DL model are combined to generate the final COT prompt for the target-specific detection samples.

[4] https://cwe.mitre.org/data/definitions/1000.html
[5] https://dwheeler.com/flawfinder
[6] http://cppcheck.net
[7] https://github.com/Yang-Yanjing/DLAP.git (COTTree).

```
{
"Static tool": vulnerability detection class,
"Vulnerability": the result of DL model
detection
}.
```

Fig. 3. A query key example of DLAP.

Step 1: Comprehending the function of the code

Step 2: Analyzing the structure of the code
- *Step* 2.1: Locating all points in the code where pointers are constructed
- *Step* 2.2: Locating all points in the code where pointers are dereferenced

Step 3: Identifying components that may introduce vulne rabilities
- *Step* 3.1: Checking for null pointer dereferences
- *Step* 3.2: Reviewing dynamic memory Management
- *Step* 3.3: Inspecting for Uninitialized Pointers
- *Step* 3.4: Tracing the pointer life cycle

Step 4: Inspecting for unsafe functions that could potenti ally lead to vulnerabilities.
- *Step* 4.1: Reviewing usage of function pointers
- *Step* 4.2: Identifying risky functions

Step 5: Integrating the information acquired above, judge whether there are potential vulnerabilities step by step

Fig. 4. An example of the refined COT prompts of DLAP.

3.5. Prompts synergy

Fig. 5 shows an example of our algorithm where the final prompts are made in such a format. The process of DLAP generating prompts is described in Algorithm 1, which takes the selected DL models, the training/history data of target detection project $\mathcal{X} = \{(x_i, y_i)\}_{i=1}^{N}$, the selected static tools $\mathcal{S}$, and the preset COT library $\mathcal{S}$ as the input.

First, the target detection project X is sampled to construct training sets $\mathcal{X}_o^T$. $\mathcal{X}_o^T$ (if open-source information is directly collected) is labeled as Y. The remaining part of detecting project X is test sets $\mathcal{X}_o^E$. $\mathcal{X}_o^T$ and label Y are utilized to minimize the loss function $\mathcal{L}$ for training $\mathcal{M}$. Next, for each input code in the test set $\mathcal{X}_o^E$, the LSH algorithm is used to find the most similar code $[Candidates]$ in this set. Then, the DL model $\mathcal{M}$ is utilized to get detection probabilities $[Probabilities]$. Using $[Candidates]$ and $[Probabilities]$, DLAP constructs question-answering combinations to form the DL-augmented ICL prompts.

DLAP carries out a bespoke process described in Fig. 5 to get the specific COT prompt. Statics tools are utilized to get analysis results $\mathcal{S}(\mathcal{X}_o^E) = \{S_1 : \text{scores}, S_2 : \text{scores}, S_3 : \text{scores}, \ldots\}$ and $Result_{\mathcal{S}}(x_i)$ is obtained by ranking $\mathcal{S}(\mathcal{X}_o^E)$. Then, DLAP generates the DL model prediction result. After that, the results $Predictions_{\mathcal{M}}$ are combined to form a query, which is utilized as the key to retrieve COT prompts generation guidance $G(x_i)$ from the COT library. GPT is used to complete $G(x_i)$ to generate $cot(x_i)$ for each detection sample.

Finally, the final prompts of DLAP are composed of the COT prompts and ICL prompts. As shown in the right part of Fig. 5, we utilized superICL prompts to obtain LLM prediction (the blue part in the final prompts) by employing the completion of LLMs. Subsequently, we generated specific review steps 1-N (the yellow part in the final prompts)

Algorithm 1 Bespoke prompts for detection samples

Require: $\mathcal{M}$; $\mathcal{X} = \{(x_i, y_i)\}_{i=1}^{N}$; $\mathcal{S}$; $\mathcal{K}$.
1: $\mathcal{X}_o^T \leftarrow \text{Sample}(\mathcal{X})$ # the training set for constructing the DL-based model
   $Y \leftarrow \text{Label}(\mathcal{X}_o^T)$
   $\mathcal{X}_o^E = \mathcal{X} - \mathcal{X}_o^T$
2: $\mathcal{M} = \text{argmin}\,\mathcal{L}(\mathcal{X}_o^T, Y)$
3: **for** $x_i, i = 1$ to N in $\mathcal{X}_o^E$ **do**
4: $[Candidates] = LSH(x_i)$
5: $[Probabilities] = \mathcal{M}([Candidates])$
6: ICL prompt$(x_i) = \{[Candidates], [Probabilities]\}$
7: $Result_{\mathcal{S}}(x_i) \leftarrow \text{Ranking}(\mathcal{S}(\mathcal{X}_o^E))$
8: $\mathcal{P}(x_i) = Prediction_{\mathcal{M}}(x_i) + Result_{\mathcal{S}}(x_i)$
9: Utilizing $\mathcal{P}(x_i)$ as the key to retrieve COT prompts generation guidance. $G(x_i) = \mathcal{K}(\mathcal{P}(x_i))$
10: Using GPT to complete $G(x_i)$: $COT(x_i) = GPT(G(x_i))$
11: DLAP prompts(x_i) = ICL prompt(x_i) + COT prompt(x_i)
12: **end for**
Ensure: Specific COT prompts of all detection samples.

by completing Bespoke COT Guidance. We concatenated the prompts in a specific sequence with natural language and finally configured the format for the LLM to organize the information. Fig. 6 illustrates an example of final prompts. The LLM then delivered the final detection judgment based on the final prompts, following the specified format and providing the judgment reason as the auxiliary information. The content with the same color in Figs. 5 and 6 corresponds to each other.

4. Experimental design

This section details research questions, datasets, DL models, baseline LLM prompts, and evaluation metrics.

4.1. Research questions

The experimental evaluation of DLAP is structured with three research questions (RQs).

RQ1: Which category of DL models is the most effective for DLAP?

Motivation&Setup: An important driver of DLAP is the DL model, which adds the information stored by the DL model training to the prompting process of LLMs through the ICL approach. As little is known about which category of DL models is suitable for augmenting LLMs in the context of vulnerability detection, we propose RQ1 to compare three representative DL models: Sysevr, Devign, and Linevul (cf. Section 4.3 for rationales).

RQ2: How effective is DLAP compared to existing prompting frameworks?

Motivation&Setup: Previous research has shown that the performance of LLMs is susceptible to prompts and inappropriate prompts lead to unsatisfactory performance. In this paper, we design DLAP as a prompt augmented framework for vulnerability detection. In RQ2, we compare DLAP against four existing prompting frameworks: $\mathbf{P}_{\text{Rol}}$, $\mathbf{P}_{\text{Aux}}$, $\mathbf{P}_{\text{Cot}}$, GRACE (cf. Section 4.4 for rationales) to evaluate its effectiveness.

RQ3: How effective is DLAP compared to LoRA fine-tuning?

Motivation&Setup: Previous research has shown that fine-tuning is helpful for augmenting LLMs. In this paper, we use prompt engineering rather than fine-tuning to develop DLAP because of its lower

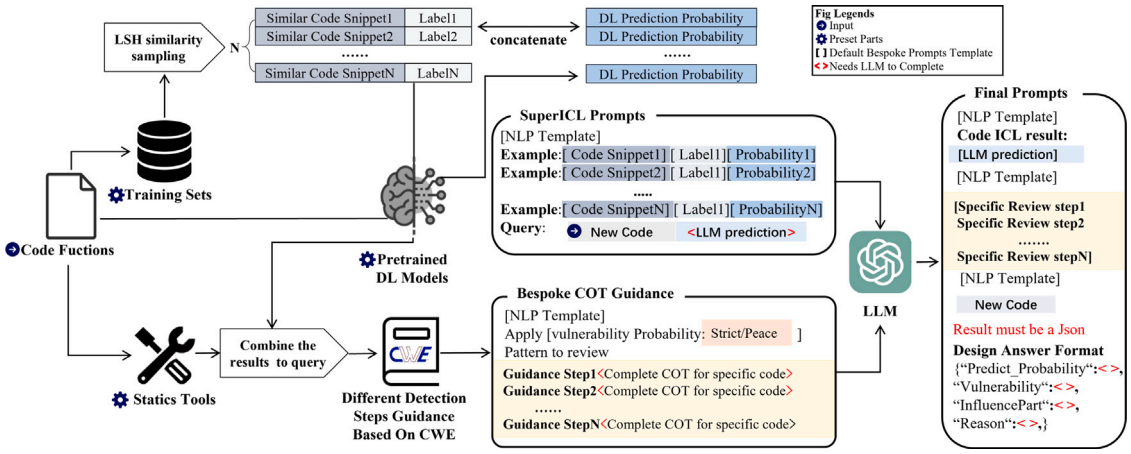

Fig. 5. The final prompts formed by the collaboration of the two parts in the DLAP.

Final Prompts	
Code	The code need to be detected
NLP Template	The In-context learning provides.
LLM Prediction	There is **0.997534565** (ICL result) confidence that it is vulne -rable.
NLP Template	Here is some review steps.
COT Steps	1. The function nfs4_atomic_open .. 2. It checks if the lookup_create flag.. 3. It then looks up the rpc credentials and proceeds to open.. 4. …(Other steps)
NLP Template	Then you should follow the review step to ma--ke the final an swer with reason.
Format Set	Your final answer must only is one json text st-ring without a ny reason and anothor texts.
Json Format Set	Josn format{ "label": "1"(vulnerability), "confidence": your final confidence (between 0-1) for label 1, "vulnerability":only answer yes or no, "influence Components":answer example (buffer or pointer o r something else), "reason":the reason for answers. }

Fig. 6. An example of specific final prompts of DLAP.

cost. In RQ3, we compare DLAP against a fine-tuning LLM (Llama-13B) (Touvron et al., 2023) to see if it has the same performance as fine-tuning. Specifically, we select LoRA (Hu et al., 2022), a state-of-the-art LLM fine-tuning technique for comparison.

4.2. Datasets

According to Croft et al. (2023), the common vulnerability detection datasets have labeling bias. To develop an appropriate experimental dataset, we customize three criteria for selecting projects: (1) it has been researched by related work (Chakraborty et al., 2022; Fan et al., 2020; Zhou et al., 2019) (to ensure external validity); (2) it has accumulated more than 3,000 functions (to exclude no-active projects); and (3) it is traceable (to exclude projects whose vulnerability information is incorrect or even unknown). As a result, our experimental dataset consists of four open-source projects, including Chrome, Linux, Android, and Qemu. These projects we selected are of good open-source quality and have high-quality vulnerability fix records for traceability.

The basic information of the selected projects is shown in Table 1, from which we can observe the number of trues (vulnerabilities) and falses in each project is imbalanced. To mitigate the impact of data

Table 1
Basic information of dataset.

Project	#Functions	#Vulnerbilities	Used by
Chrome	77,173	3939	Chakraborty et al. (2022)
Linux	46,855	1961	Fan et al. (2020)
Android	8691	1277	Fan et al. (2020)
Qemu	3096	125	Zhou et al. (2019)

imbalance on training the DL augment model, we first performed random undersampling on the non-vulnerable samples of the four projects. Then we divided the dataset into training and testing sets with the 8:2 proportion. The training set was used to build DL models, while the testing set was used to evaluate the performance of DLAP.

4.3. DLAP refinement

To address RQ1, we select three DL models for vulnerability detection to refine DLAP. Each of the three represents one type of DL model. Their rationales and hyperparameter settings are as follows.

- **Sysevr** (Li et al., 2021) represents the category that uses code features including syntactic, semantic, and vector representation. It filters the code into slice input by static analysis of semantics and syntax.
- **Devign** (Zhou et al., 2019) represents the category that introduces more structured graph structures and graph neural networks into the vulnerability detection model.
- **Linevul** (Fu and Tantithamthavorn, 2022) represents the category that utilizes a pre-trained deep learning model. This novel system detection model is based on the Transformer architecture.

We selected these three DL models to evaluate a range of similar models for the categories they each represent. As part of our model selection process, we referenced the parameters reported in the respective research papers of these DL models that achieved the best performance. These parameters were selected in our framework in Table 2 as the pre-set hyperparameters. By doing so, we aim to replicate the optimal performance achieved by these models and ensure consistency in our evaluation and comparison.

4.4. Baselines

We compare DLAP against four prompting frameworks (Zhang et al., 2023c; Purba et al., 2023; Lu et al., 2024; White et al., 2023) that leverage LLMs to detect vulnerabilities.

Table 2
The settings of DL model.

DL model	Hyperparameter	Selection
Sysevr	Java version	Java 8
	Static tools	Joern 0.3.1
	Graph database	Neo4j
	Data preprocessing	Slice
	Embedding algorithm	**Word2vec**
	-sampling algorithm	CBOW
	-sampling window	5
	-min_Count	5
	Network architecture	**BiLSTM**
	-epoch	100
	-batch_size	32
	-optimizer	sgd
	-loss function	binary cross-entropy
Devign	Java version	Java 8
	Static tools	Joern 2.0.157
	Data preprocessing	Graph
	Embedding algorithm	**Word2vec**
	-verctor_size	100
	-epoch	10
	-min_count	1
	Network architecture	**GNN+CNN**
	-epoch	200
	-batch_size	128
	-input_channels	115
	-hidden_channels	200
	-num_of_layers	6
	-optimizer	adam
	-loss function	binary cross-entropy
Linevul	Data preprocessing	Slice
	Embedding algorithm	BPE+Transformer
	Pretrained model	**codeBERT**
	-batch_size	256
	-num_attention_head	12
	-optimizer	Adam
	-loss function	binary cross-entropy

- $\mathbf{P}_{\text{Rol}}$ (Role-based prompts): According to White et al. (2023), providing GPT with a clear role would greatly alleviate its illusion problem. Our first baseline is proposed by Zhang et al. (2023c), making GPT a vulnerability detection system.

Role-based prompts

I want you to act as a **Vulnerability Detection System**. My first request is "Is the following program buggy?" Please answer Yes or No. [CODE]

- $\mathbf{P}_{\text{Aux}}$ (Auxiliary information prompts): Based on the view of Zhang et al. (2023c), providing the LLMs more semantic information about the code for vulnerability detection improves its performance. Therefore, in baseline 2, we provide data flow as auxiliary information to prompts.

Auxiliary information prompts

I want you to act as a vulnerability detection system. I will provide you with the original program and the data flow information, and you will act upon them. Is the following program buggy? [CODE], [DF description].

- $\mathbf{P}_{\text{Cot}}$ (Chain-of-thought prompts): According to Wei et al. (2022), due to the potential capabilities of LLMs for multi-turn dialogue, constructing a COT better assists LLMs in reasoning (Wei et al., 2022). Therefore, in baseline 3, we constructed a two-step thinking chain to drive the LLM in the process of vulnerability detection. `Step1`: To make LLMs correctly determine whether a code is vulnerable. This step drives the LLMs to understand the exact purpose of the code. Therefore, we designed first-step prompts to detect the intent of the code. `Step2`: Based on the first step, we continue to prompt LLMs to detect vulnerabilities for inputs.

Chain-of-thoughts prompts

`Step1`: Please describe the intent of the given code. [CODE].
`Step2`: I want you to act as a vulnerability detection system. Is the above program buggy? Please answer Yes or No

- **GRACE**: GRACE is a vulnerability detection prompting framework that enhances the capabilities of LLM for software vulnerability detection. It achieves this by incorporating graph structural information from the code. GRACE employs codeT5 and ICL techniques to use graph information.

4.5. LLMs selection

When comparing different prompt engineering methods, we selected GPT-3.5-turbo-0125 as the prompting-driven LLM due to its minimal usage restrictions and well performance. GPT-3.5-turbo has also been utilized in peer-reviewed baseline-related work (Lu et al., 2024; Sun et al., 2024).

To address RQ3, we developed five criteria to select the suitable LLM for further fine-tuning:

- c1: The LLM has been widely used for fine-tuning tasks in related work.
- c2: The LLM has demonstrated its adaptability with LoRA.
- c3: The LLM has a comparable capability with GPT-3.5-turbo.
- c4: The LLM can be deployed and modified locally.
- c5: The attention layer parameters of the LLM can be calculated.

Following the selection criteria, we first reviewed related work and found Vicuna-13B, a model fine-tuned from LLama-13B was optimal. LLama-13B has been widely used in related work (c1) and has demonstrated excellent performance when fine-tuned with LoRA (c2) (Lu et al., 2023; Ince et al., 2024; Hossen et al., 2024). According to Zheng et al. (2024), Vicuna-13B fine-tuned from LLama-13B can achieve 90% performance of GPT-series on various benchmark tasks, with fewer parameters (c3). Vicuna-13B was suitable because it could be deployed and modified locally (c4), which makes it possible to calculate the attention layer parameters of LLMs (c5). Therefore, the internal attention layer parameter distribution of an LLM could be analyzed and compared with other methods (*e.g.*, DLAP in this paper).

In RQ3, we acknowledge that comparing GPT and local LLMs augmented by the DLAP framework is natural. However, we did not compare GPT and local LLMs due to technical infeasibility. It has been widely demonstrated that ICL and COT prompts cannot work effectively on LLMs with fewer parameters (Arakelyan et al., 2023). Following studies (Wu et al., 2023; Zhou et al., 2024; Pornprasit and Tantithamthavorn, 2024) that are relevant to comparing prompting and fine-tuning, we compare GPT (augmented by DLAP prompting) to Vicuna-13B (fine-tuned by LoRA). Vicuna-13B is selected through the criteria presented below.

4.6. Evaluation metrics

As vulnerability detection is formulated as a binary classification problem in this paper, we use precision ($\mathcal{P}_{\text{vul}}$), recall ($\mathcal{R}_{\text{vul}}$), and F1-score (F_1) to measure the performance of each framework. Considering vulnerability is a minor class but is of great severity, we also use FPR as a metric. FPR pays attention to false positives since making mistakes on them would cause more serious outcomes than making mistakes on false negatives. In this paper, the minor class (positive) is vulnerability,

Table 3
Results of DL model comparison.

Project	Linevul					Devign					Sysevr				
	$\mathcal{P}_{vul}$	$\mathcal{R}_{vul}$	$F_1 vul$	FPR	MCC	$\mathcal{P}_{vul}$	$\mathcal{R}_{vul}$	$F_1 vul$	FPR	MCC	$\mathcal{P}_{vul}$	$\mathcal{R}_{vul}$	$F_1 vul$	FPR	MCC
Chrome	**40.4**	73.3	**52.1**	**28.4**	**37.6**	29.3	**85.5**	43.7	54.0	26.1	27.7	56.8	37.2	39.0	14.6
Android	**34.6**	**86.2**	**49.3**	**41.4**	**36.1**	31.7	85.5	46.2	46.7	31.3	29.4	80.3	43.1	48.7	25.5
Linux	**57.1**	**76.4**	**65.4**	**13.9**	**56.4**	48.8	66.3	56.3	16.9	44.4	27.7	22.6	24.9	14.4	08.8
Qemu	**84.2**	55.1	**66.7**	**01.9**	**63.9**	52.8	**65.5**	58.5	10.7	50.3	28.6	10.0	14.8	04.4	09.0

occupying a very small portion. The definition of FPR is shown in Eq. (6). Moreover, the Matthews correlation coefficient (MCC) is also used as an evaluation metric. MCC, a.k.a. phi coefficient, is a metric to measure the performance of binary classifiers on imbalanced datasets. MCC is a more comprehensive metric than FPR. The definition of MCC is shown in Eq. (7).

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} \tag{6}$$

$$\text{MCC} = \frac{\text{TP} \times \text{TN} - \text{FP} \times \text{FN}}{\sqrt{(\text{TP} + \text{FP})(\text{TP} + \text{FN})(\text{TN} + \text{FP})(\text{TN} + \text{FN})}} \tag{7}$$

where TP represents correctly detected vulnerabilities, TN represents correctly detected non-vulnerabilities, FP represents incorrectly detected vulnerabilities, and FN represents incorrectly detected non-vulnerabilities.

The Coefficient of Variation (CV) is a statistical measure used to determine the dispersion of data points in a dataset relative to its mean. It is particularly valuable when comparing the variability of datasets with different means. The CV is calculated using Eq. (8):

$$\text{CV} = \frac{\sigma}{\mu} \tag{8}$$

where σ represents the standard deviation and μ denotes the mean of the dataset.

A higher CV indicates greater dispersion within the data distribution, reflecting more variability relative to the mean. $\mathcal{P}_{vul}$, $\mathcal{R}_{vul}$, F_1, and FPR range from 0 to 1, with higher values indicating better performance of a classifier. MCC ranges from −1 to +1, with higher values indicating better classifier performance. We use **percentage values** (%) to highlight the differences between results.

5. Results and analysis

This section analyzes the experimental results to address the research questions.

5.1. RQ1: Selection of DL models

To investigate which category of DL model is the most suitable for DLAP, we conducted experiments on four large-scale projects following these steps. First, we observed the augmenting effect on LLM by directly integrating these pre-trained DL models. Second, we analyzed the discrete distribution of the probability levels of the outputs from the DL models. Finally, we selected the pre-trained DL model demonstrating the best augmenting effect and the highest degree of output probability dispersion.

The results are provided in Table 3, which reveals that using Linevul outperforms using others in most datasets and metrics. For instance, in the Chrome dataset, DLAP with Linevul achieves the highest MCC of 37.6%, surpassing Devign's 26.1% and Sysevr's 14.6%. This finding is consistent in the Linux dataset, where it secures an MCC of 56.4%, compared to 44.4% and 8.8% for Devign and Sysevr, respectively. Furthermore, the precision and F1 scores of Linevul are notably higher across the datasets, underscoring its robustness in identifying vulnerabilities with greater accuracy and fewer false positives, as evidenced by its lower FPR in datasets. Overall, using Linevul surpasses using Devign by an average of 7.2% and 10.5% on the comprehensive evaluation metrics F1 and MCC, respectively. It also outperforms integrating Sysevr by an average of 28.4% and 34.0% on the same metrics. This demonstrates that Linevul has superior adaptability and generalizability when integrated into LLMs compared to other DL models. These results indicate the effectiveness of integrating Linevul into DLAP to detect vulnerabilities, especially its superior F_1, which implies a higher likelihood of detecting actual vulnerabilities. Its MCC, a critical indicator of the quality of binary classifications, shows the ability of DLAP with Linevul to solve extremely imbalanced datasets.

Table 4
CV of DL model comparison.

DL model	Chrome	Android	Linux	Qemu	Average
Sysevr	0.1	1.2	0.4	0.02	0.43
Devign	0.5	1.2	2.0	2.0	1.4
Linevul	**2.4**	**2.6**	**2.5**	**3.3**	**2.7**

To further distinguish which DL model is more suitable as a plug-in for DLAP, we also analyze the intermediate output (detection probability) of the DL model. Table 4 presents the variability in the performance of different DL models across several software projects. The Linevul model, highlighted in gray and bold, displays the highest CV. By comparing and analyzing probability density distribution plots Fig. 7 and CV Table 4 in the largest project dataset Google, we noticed that Linevul displays a more discrete data distribution when compared to the other models. This unique discrete detection distribution property facilitates LLM generation with implicit fine-tuning for downstream detection tasks more effectively.

Summary for RQ1: Linevul achieves the best performance among the three types of DL models. Moreover, Linevul results in the most discrete detection probability, indicating its prediction has the highest confidence so that it can stimulate LLMs best. Therefore, we select Linevul as the driver of DLAP to conduct the follow-up experiments.

5.2. RQ2: Comparison with other prompting frameworks

Due to cost constraints associated with OpenAI API calls, we employed the GPT-3.5-turbo-0125 model for vulnerability detection. Table 5 illustrates the performance comparison between the GPT model using the baseline prompting framework and the DLAP approach. The performance of each framework is evaluated based on five metrics: Precision ($\mathcal{P}_{vul}$), Recall ($\mathcal{R}_{vul}$), F1 Score (F_1), False Positive Rate (FPR), and Matthews Correlation Coefficient (MCC). DLAP consistently outperforms the other frameworks across nearly all metrics and datasets. Specifically, DLAP achieves the highest Precision, Recall, F1 Score, and MCC values, showcasing its superior ability to accurately identify vulnerabilities with minimal false positives. For instance, in the Chrome dataset, DLAP's Precision of 40.4% and Recall of 73.3% significantly surpass those of the next best framework, GRACE. Furthermore, DLAP's exceptional performance is highlighted by its F1 Score, reaching up to 52.1% in Chrome, 49.3% in Android, 65.4% in Linux, and an impressive 66.7% in Qemu, which are higher compared to the baseline frameworks. In terms of FPR, DLAP demonstrates a moderate FPR across the datasets. Despite that FPR is not the lowest on the Chrome, Android, and Linux datasets when compared to the baselines of $\mathbf{P}_{\text{Rol}}$

Table 5
Results of prompting framework comparison.

Framework	Chrome					Android					Linux					Qemu				
	$\mathcal{P}_{vul}$	$\mathcal{R}_{vul}$	$F_1 vul$	FPR	MCC	$\mathcal{P}_{vul}$	$\mathcal{R}_{vul}$	$F_1 vul$	FPR	MCC	$\mathcal{P}_{vul}$	$\mathcal{R}_{vul}$	$F_1 vul$	FPR	MCC	$\mathcal{P}_{vul}$	$\mathcal{R}_{vul}$	$F_1 vul$	FPR	MCC
P_{RoI}	24.4	07.2	11.1	**05.8**	02.3	22.5	06.4	10.0	05.6	01.3	22.4	06.6	10.2	05.6	01.7	22.2	06.9	10.5	04.4	04.2
P_{Aux}	22.7	54.6	32.1	48.6	04.8	21.8	63.4	32.5	58.3	04.2	24.6	70.2	36.5	52.6	14.1	19.3	55.2	28.6	42.1	09.5
P_{Cot}	16.8	05.4	08.1	07.0	02.6	31.6	03.1	05.7	**01.7**	04.0	30.7	08.0	12.7	**04.4**	06.5	64.7	38.0	47.8	03.8	43.0
GRACE	32.6	37.5	32.6	80.2	11.2	25.0	82.6	38.4	74.0	08.5	25.0	76.0	37.6	76.0	02.0	17.1	**93.1**	28.9	82.4	10.6
DLAP	**40.4**	**73.3**	**52.1**	28.4	**37.6**	**34.6**	**86.2**	**49.3**	41.4	**36.1**	**57.1**	**76.4**	**65.4**	13.9	**56.4**	**84.2**	55.1	**66.7**	**01.9**	**63.9**

Table 6
Results of performance comparison with fine-tuning.

Dataset	Fine-Tuning Vicuna-13B					DLAP				
	$\mathcal{P}_{vul}$	$\mathcal{R}_{vul}$	$F_1 vul$	FPR	MCC	$\mathcal{P}_{vul}$	$\mathcal{R}_{vul}$	$F_1 vul$	FPR	MCC
Chrome	91.4	74.4	82.0	01.8	78.6	40.4	73.3	52.1	28.4	37.6
Android	67.0	35.8	46.7	04.5	40.4	34.6	86.2	49.3	41.4	36.0
Linux	96.4	55.4	70.3	00.5	68.9	57.1	76.4	65.4	14.0	56.4
Qemu	99.9	06.7	12.1	00.1	23.4	84.2	55.2	66.7	01.9	63.9
Total	**88.7**	43.0	52.8	**01.2**	**52.8**	54.1	**72.8**	**58.4**	21.4	48.5

and $\mathbf{P}_{Cot}$, DLAP is far superior to them on the F_1 and MCC. Therefore, DLAP's overall effect exceeds the baseline frameworks.

In particular, DLAP's MCC values, which indicate the quality of binary classifications, significantly exceed those of the other methods, such as 37.6% in Chrome and 63.9% in Qemu, further establishing its superior performance in the task of vulnerability detection using LLMs. Our framework consistently surpasses the top baseline in terms of the MCC indicator, which, based on the nature of the MCC correlation coefficient, suggests that our predictions more accurately reflect the actual distribution and indicates DLAP is superior to the baselines in the generalization performance of large data sets.

Overall, the analysis reveals that DLAP not only excels in identifying vulnerabilities with high precision and recall but also maintains a low false positive rate and achieves outstanding overall performance as evidenced by its F1 Scores and MCC values. This demonstrates DLAP's exceptional effectiveness in harnessing the power of LLM for the critical task of vulnerability detection, which outperforms the capabilities of other prompting frameworks.

Summary for RQ2: DLAP's overall performance is superior to other prompting frameworks, which is evidenced by its exceptional MCC scores and higher values in $\mathcal{P}_{vul}$, $\mathcal{R}_{vul}$, and $F_1 vul$.

5.3. RQ3: Prompting vs. Fine-tuning

Table 6 shows that fine-tuning an LLM on a large project has a higher F_1 than DLAP. However, on a small project with imbalanced data, DLAP performs better. In particular, LLMs cannot fine-tuned on Qemu because the project has a small amount of data. In contrast, DLAP gets the distribution characteristics of small samples and hence can achieve better performance. In addition, fine-tuning an LLM requires stopping the model and retraining it before using it, whereas DLAP does not need to be removed for retraining during its use. It is used as a plug-in to access an LLM in real time to augment the vulnerability detection capability of LLMs. Besides, the comparison of computational cost between DLAP and LoRA fine-tuning is shown on Table 7. It is clear that fine-tuning a 13B LLM requires close to 40 GB of graphics memory and a lot of time. In contrast, DLAP can select a small DL model and train it to fit the target data in less than one hour.

Eq. (5) (Cf. Section 3.3) indicates that DL model training information changes the relaxed attention of the LLM. This results in an implicit fine-tuning for the LLM to adapt to a specific detection task (Dai et al., 2023). Whether through fine-tuning or In-Context Learning (ICL), the extent to which a model is fine-tuned reflects its ability to adapt to the target task, serving as a crucial factor in stimulating LLMs to perform well. According to the detection result shown in Table 6, DLAP performs well in approximating fine-tuning on performance evaluation metrics.

Table 7
Results of computational cost comparison.

Dataset	Fine-Tuning Vicuna-13B			DLAP		
	M(MB)	T(h)	GPU(GB)	M(MB)	T(h)	GPU(GB)
Chrome	5.1	11.1	31.2	3.6	0.8	6.3
Android	4.9	4.2	30.3	4.3	0.5	5.5
Linux	4.9	5.5	30.3	3.8	0.4	5.5
Qemu	4.8	1.3	28.7	0.9	0.3	2.8

To further explain what mechanism induces LLM to produce implicit fine-tuning and achieve good performance on the target task, we extract the attention layer from the fine-tuned local LLM to calculate the probability for each detection category. Subsequently, we gather the ICL outputs by the LLM with DLAP to calculate the probability for each detection category. The probability distribution of the different classes indicates the degree of fine-tuning of the model. Fig. 8 shows that probability distributions between fine-tuning and DLAP are similar. The same distribution explains that DLAP enables implicit fine-tuning at a reduced cost.

In comparison with fine-tuning, Fig. 9 shows a real example of using DLAP to detect vulnerabilities in Linux. The outcomes, which are easily understandable to developers, closely match the records from the actual issue fix commit. In contrast, the output from a fine-tuned LLM is limited to simple 'yes' or 'no' responses. Fine-tuning pre-trained LLMs for specific tasks causes catastrophic forgetting as it changes the model's internal weights. Parisi et al. (2019), Masana et al. (2022). Vulnerability detection is a binary classification task. It worsens catastrophic forgetting because the task labeling is too simplistic. Additionally, the optimization goals do not align with the model's pre-training tasks. Fine-tuned LLMs perform well in vulnerability detection but can become binary classifiers. They may only answer "yes" or "no", losing their ability to provide explanations. However, as a form of prompt engineering, DLAP does not modify the weights of LLMs. Thus, It is not susceptible to catastrophic forgetting. DLAP results with causes for vulnerabilities are clearer to developers than fine-tuning results.

Summary for RQ3: Although the overall performance of DLAP is slightly lower than that of fine-tuning, their predictive distributions are similar. This result shows that DLAP prompts LLMs to produce effective implicit fine-tuning with performance comparable to that of explicit fine-tuning but at a significantly lower cost. DLAP can provide causes for detection results, making them more comprehensible to developers.

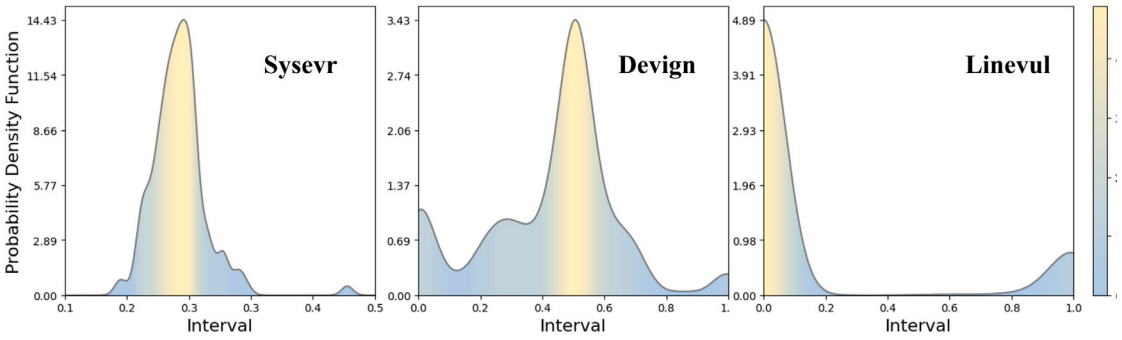

Fig. 7. Distribution of probability density (Sysevr, Devign, and Linevul) on Google project.

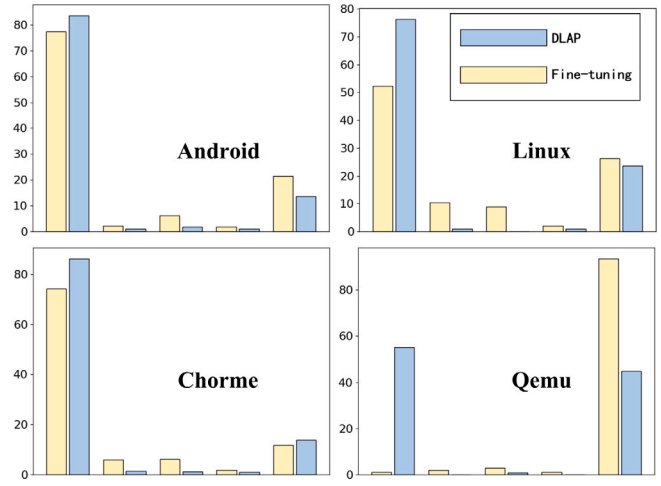

Fig. 8. Predictive distributions resulted by DLAP and fine-tuning.

6. Discussion

In this section, we discuss the DL model selection for DLAP and DLAP's potential generalization capability.

6.1. DL model selection for DLAP

Based on the insights gained from RQ1, DL models with discrete predictive probability density distributions for the data are more suitable as an integrated plug-in for DLAP. Additionally, we have observed the effectiveness of a DL model as an LLM prompt model. We have discovered that its utility significantly improves when it exhibits discrete data with the highest value of CV. Moreover, our experiments have highlighted the exceptional performance of Transformer-based models in driving the LLM. This advantage could be attributed to the architectural resemblance between Transformer models and the design architecture of the LLM. The similarity in their structures allows for seamless integration, enabling the attention-layer parameters derived from Transformer models to play a pivotal role in facilitating implicit fine-tuning within the LLM. By leveraging these attention-layer parameters, the LLM dynamically adjusts and refines its internal mechanisms, implicitly adapting itself to the nuances and intricacies of different downstream tasks. This implicit fine-tuning process empowers the LLM to generate more accurate and contextually relevant responses, thereby enhancing its overall performance in various application scenarios.

In summary, our experiments have demonstrated the importance of selecting suitable DL models for DLAP and the advantages of Linevul. Software organizations can leverage Linevul to augment LLMs as the first choice but can still seek more suitable DL models when there is a wider range of options.

6.2. Generalization capability of DLAP

The DLAP framework effectively stimulates LLMs to implicitly fine-tune themselves for other software development tasks. By integrating existing static tools and deep learning models, DLAP is applied to a variety of ASAT tasks. This adaptation simplifies the process of adopting DLAP to deal with new challenges. We introduce two scenarios that may extend the applicability of DLAP.

Automated identification of affected libraries from vulnerability data is an ASAT task that requires figuring out which libraries in software are related to each of the reported vulnerabilities in open vulnerability report sets (*e.g.*, NVD, CVE). The task is formulated as an extreme multi-label learning (Haryono et al., 2022; Chen et al., 2020). First, DLAP constructs a sufficient vulnerability description database and combines them with libraries known to be affected by the reported vulnerabilities as a COT template library for affected libraries identification. The known affected libraries are collected to train a DL model. Then, existing static tools (fastXML[8]) and the DL model are used to generate paramilitary results for the XML library list in the project. Finally, by combining the results as a key to query the COT template library, COT prompts can be built to augment the LLMs, and identifying libraries from vulnerability data may be more accurate.

Code smell detection is an ASAT task that prevents software from technical debts. Code smell detection based on DL models is a multi-classification detection task comprised of several binary classification models, each designed to detect a specific category of code smell (Lewowski and Madeyski, 2022; Pecorelli et al., 2019). Utilizing DLAP requires the creation of a comprehensive reference library of code smells and a high-quality coding standards library. Then DLAP uses the static tools (checkstyle[9]) and DL models for specific detection projects. Like the progress mentioned in this paper, employing the DL model augments the LLMs with prompts for the specific project code smell detection. DLAP is utilized in other ASAT tasks which need to combine DL models with LLMs to improve the performance of LLMs in the target tasks.

7. Threats to validity

This section analyzes possible threats to validity (Zhou et al., 2016) and our efforts to mitigate their impacts.

Internal Validity. The efficacy of the DLAP relies on its core component, the DL models. While it is permissible for these DL models to result in judgment biases for input code, a completely erroneous DL model that is either irrelevant or detrimental to the task can severely undermine the performance of the DLAP. Therefore, when employing the DLAP, it is crucial to select DL models that are capable of addressing the specific objectives of the task to effectively augment

[8] https://github.com/fastXML/fastXML
[9] https://checkstyle.org

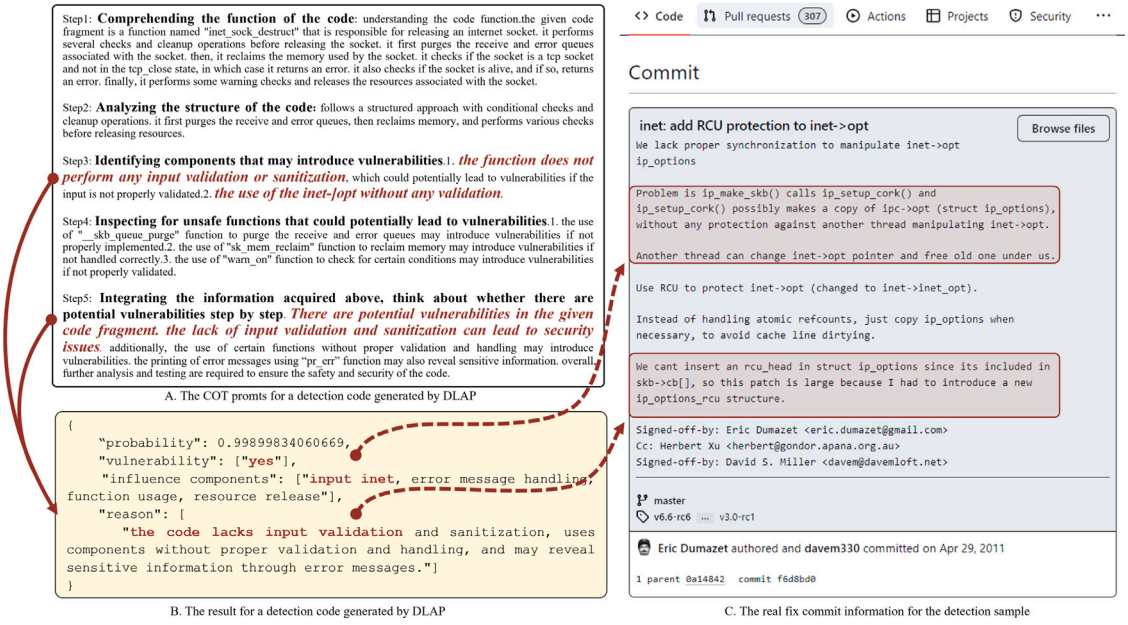

Fig. 9. An example of applying DLAP to repair Linux code issues (commit id: f6d8bd0).

the performance of LLMs. Besides, due to the close source nature of certain LLMs (GPT-3.5-turbo), their internal structures and the specific fine-tuning methods they employ remain unknown. Therefore, for our experiments, we use an open-source LLM (Llama-13b), for comparative fine-tuning studies.

Construct Validity. The relaxed attention of the LLMs changes under the stimulation of DLAP according to Eq. (5). We define this stimulation as the implicit fine-tuning of LLMs caused by DLAP to adapt to the feature of the target project. Because of the limitations of observing the internal output of LLMs, we cannot strictly demonstrate that the stimulation produces gradient descent optimization loss on the target classification task. Instead of performing a demonstration mathematically, we show some of our results with advantages and some of the intermediate outputs of some contrasts showing fine-tuning, validating the existence of the implicit fine-tuning mechanism in the way of experimental data. These visualized experimental results eliminate the construct validity of this paper to some extent.

External Validity. For the verification of DLAP, the final effect of our template needs to drive LLM to complete the vulnerability detection, and the performance of this task is used as an evaluation to measure the effectiveness of our method. So when the LLM is different from the LLM selected in this experiment, the results of using DLAP will be different. We identify the choice of LLM as an external validity threat to this work. Considering both cost and model performance, we chose the least expensive model of the current state-of-the-art LLMs, GPT-3.5-turbo-0125. By using the best model, we make the best use of DLAP. we provide specific model selection, which ensures that other work reproduces the same level of improvement when using DLAP.

Conclusion Validity. The robust performance of DLAP is related to LLMs and the pre-trained DL models. LLMs are known for their robustness in cross-project tasks. Moreover, pre-trained DL models also possess a degree of cross-project robustness. However, when the projects are not included in the training set and have significantly different data distributions, they may suffer performance degradation. To mitigate this problem, new project data could be collected and annotated to re-train the DL models that augment DLAP. Alternatively, DLAP could use a prior routing algorithm to select the most suitable pre-trained DL models (i.e., those with training set features similar to the detection samples) to eliminate the threat.

8. Conclusion

In this paper, we propose DLAP, a bespoke prompting framework for ASAT tasks that has superior and stable performance in software vulnerability detection tasks with results easily understandable to developers. Experiments show the effectiveness of augmenting LLMs by DL models to stimulate adaptive implicit fine-tuning. This progress prompts LLMs to exceed both state-of-the-art DL solutions and LLMs with alternative prompting frameworks in vulnerability detection. Through experiments, we also find that the pre-trained knowledge of LLMs combines the outputs of all parts of DLAP to achieve good performance. In the future, by extensively collecting and confirming annotations on specific vulnerability types and causes from vulnerability reports, we can evaluate the usability of method explanation text generation. Based on this evaluation, we can improve generation performance by designing a retrieval-augmented context augmentation algorithm. Furthermore, we will utilize DLAP in more ASAT tasks to explore how DLAP is generalized to the other tasks.

CRediT authorship contribution statement

Yanjing Yang: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Xin Zhou:** Writing – review & editing, Project administration, Methodology, Software, Resources, Funding acquisition, Conceptualization. **Runfeng Mao:** Methodology, Conceptualization. **Jinwei Xu:** Visualization, Validation, Software, Data curation, Conceptualization. **Lanxin Yang:** Writing – review & editing, Writing – original draft, Methodology. **Yu Zhang:** Visualization, Software, Data curation. **Haifeng Shen:** Writing – review & editing, Methodology. **He Zhang:** Writing – review & editing, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the Natural Science Foundation of Jiangsu Province , China(No. BK20241195), the National Natural Science Foundation of China (No. 62202219, No. 62072227, No. 62302210), the Jiangsu Provincial Key Research and Development Program, China (No. BE2021002-2), and the Innovation Project and Overseas Open Project of State Key Laboratory for Novel Software Technology (Nanjing University), China (ZZKT2024A18, ZZKT2024B07, KFKT2023A09, KFKT2023A10, KFKT2024A02, KFKT2024A13, KFKT2024A14).

Appendix

The appendix analyzes the composition and causes of implicit fine-tuning. The implicit fine-tuning process is described as follows. we sets $C(x) = COT(x)$ as the input representation for the context. It sets the target task data input as x, and $\mathcal{P}(x) = ICL(x)$ as the input representation for DLAP prompts query. W_Q, W_K, W_V are the projection matrices for computing the attention queries. $\mathbf{q} = W_Q$ is the attention query vector. In the fine-tuning process of DLAP, the attention of LLMs is represented as Eq. (9)

$$\begin{aligned}\mathcal{A}(\mathbf{q}) &= \text{Attention}(V, K, \mathbf{q}) \\ &= W_V[\mathcal{P}(x); C(x)]\,\text{softmax}\left(\frac{\left(W_K[\mathcal{B}(x); C(x)]\right)^T \mathbf{q}}{\sqrt{d}}\right)\end{aligned} \tag{9}$$

where W_Q, W_K, W_V are the projection matrices for computing the attention queries. To simplify this process, we eliminate the nonlinear function softmax and related scaling factor $\sqrt{d}$ to facilitate the analysis of the attention changes in this process itself. We obtain an approximate relaxed linear attention, Eq. (10).

$$\begin{aligned}\mathcal{A}(\mathbf{q}) &\approx W_V[\mathcal{P}(x); C(x)]\left(W_K[\mathcal{P}(x); C(x)]\right)^T \mathbf{q} \\ &= W_V C(x)\left(W_K C(x)\right)^T \mathbf{q} + W_V \mathcal{P}(x)\left(W_K \mathcal{P}(x)\right)^T \mathbf{q} \\ &= \widetilde{\mathcal{A}}(\mathbf{q})\end{aligned} \tag{10}$$

We define the context (information from COT library) prompts as the initial parameters W_{init} that need to be updated by the attention layer in Eq. (11).

$$W_{\text{init}} = W_v[C(x)] \cdot W_K[C(x)]^T \cdot \mathbf{q}) \tag{11}$$

According to research (Dai et al., 2023), we reverse use the dual form of the attention of transformer derived by them. Therefore, the adaptive implicit fine-tuning of LLMs stimulated by DLAP for specific projects are written as Eq. (10)

$$\begin{aligned}\widetilde{\mathcal{A}}(\mathbf{q}) &= W_{\text{init}}\mathbf{q} + W_V[\mathcal{P}(x)]\left(W_K[\mathcal{P}(x)]\right)^T \mathbf{q} \\ &= W_{\text{init}}\mathbf{q} + \text{LinearAttn}\left(W_V[\mathcal{P}(x)], W_K[\mathcal{P}(x)], \mathbf{q}\right) \\ &= W_{\text{init}}\mathbf{q} + \sum_i \left((W_V[\mathcal{P}(x)]_i) \otimes \left(W_K[\mathcal{P}(x)]_i\right)\right)\mathbf{q} \\ &= W_{\text{init}}\mathbf{q} + \Delta W_{\mathcal{P}(x)}\mathbf{q} \\ &= \left(W_{\text{init}} + \Delta W_{\mathcal{P}(x)}\right)\mathbf{q}\end{aligned} \tag{12}$$

Through Eq. (12), we conclude that the relaxed attention mechanism is influenced by the prompt $\mathcal{P}$.

Data availability

All source code and datasets used in this study are open source. The source code of DLAP and datasets are released at: https://github.com/Yang-Yanjing/DLAP.git.

References

Arakelyan, S., Das, R., Mao, Y., Ren, X., 2023. Exploring distributional shifts in large language models for code analysis. In: Proceedings of the 22nd Conference on Empirical Methods in Natural Language Processing. EMNLP, ACL, pp. 16298–16314.

Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., et al., 2022. Constitutional ai: Harmlessness from ai feedback. arXiv preprint arXiv:2212.08073.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al., 2020. Language models are few-shot learners. Adv. Neural Inf. Process. Syst. 33, 1877–1901.

Chakraborty, S., Krishna, R., Ding, Y., Ray, B., 2022. Deep learning based vulnerability detection: Are we there yet. IEEE Trans. Softw. Eng. 48 (9), 3280–3296.

Chen, Y., Santosa, A.E., Sharma, A., Lo, D., 2020. Automated identification of libraries from vulnerability data. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Practice. SEIP, ACM, pp. 90–99.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.d., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al., 2021. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374.

Cheshkov, A., Zadorozhny, P., Levichev, R., 2023. Evaluation of ChatGPT model for vulnerability detection. arXiv preprint arXiv:2304.07232.

Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H.W., Sutton, C., Gehrmann, S., et al., 2023. Palm: Scaling language modeling with pathways. J. Mach. Learn. Res. 24 (240), 1–113.

Christakis, M., Bird, C., 2016. What developers want and need from program analysis: An empirical study. In: Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering. ASE, ACM, pp. 332–343.

Croft, R., Babar, M.A., Kholoosi, M.M., 2023. Data quality for software vulnerability datasets. In: Proceedings of the 45th IEEE/ACM International Conference on Software Engineering. ICSE, IEEE, pp. 121–133.

Dai, D., Sun, Y., Dong, L., Hao, Y., Ma, S., Sui, Z., Wei, F., 2023. Why can GPT learn in-context? Language models implicitly perform gradient descent as meta-optimizers. In: Proceedings of the 2023 ICLR Workshop on Mathematical and Empirical Understanding of Foundation Models (ME-FoMo). Association for Computational Linguistics, pp. 4005–4019.

Fan, J., Li, Y., Wang, S., Nguyen, T.N., 2020. A C/C++ code vulnerability dataset with code changes and CVE summaries. In: Proceedings of the 17th IEEE/ACM International Conference on Mining Software Repositories. MSR, ACM, pp. 508–512.

Fu, M., Tantithamthavorn, C., 2022. Linevul: A transformer-based line-level vulnerability prediction. In: Proceedings of the 19th IEEE/ACM International Conference on on Mining Software Repositories. MSR, ACM, pp. 608–620.

Gonzalez, D., Zimmermann, T., Godefroid, P., Schäfer, M., 2021. Anomalicious: Automated detection of anomalous and potentially malicious commits on github. In: Proceedings of 43rd IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP). IEEE, pp. 258–267.

Haryono, S.A., Kang, H.J., Sharma, A., Sharma, A., Santosa, A., Yi, A.M., Lo, D., 2022. Automated identification of libraries from vulnerability data: Can we do better? In: Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension. ICPC, ACM, pp. 178–189.

Hossen, M.I., Zhang, J., Cao, Y., Hei, X., 2024. Assessing cybersecurity vulnerabilities in code large language models. arXiv preprint arXiv:2404.18567.

Hsieh, Y.-G., Niu, G., Sugiyama, M., 2019. Classification from positive, unlabeled and biased negative data. In: Proceedings of the 36th ACM International Conference on Machine Learning. ICML, PMLR, pp. 2820–2829.

Hu, E.J., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al., 2022. Lora: Low-rank adaptation of large language models. In: International Conference on Learning Representations. ICLR.

Ince, P., Luo, X., Yu, J., Liu, J.K., Du, X., 2024. Detect llama-finding vulnerabilities in smart contracts using large language models. In: Australasian Conference on Information Security and Privacy. Springer, pp. 424–443.

Jin, M., Shahriar, S., Tufano, M., Shi, X., Lu, S., Sundaresan, N., Svyatkovskiy, A., 2023. Inferfix: End-to-end program repair with llms. arXiv preprint arXiv:2303.07263.

Kang, H.J., Aw, K.L., Lo, D., 2022. Detecting false alarms from automatic static analysis tools: How far are we? In: Proceedings of the 44th IEEE/ACM International Conference on Software Engineering. ICSE, ACM, pp. 698–709.

Katsadouros, E., Patrikakis, C.Z., Hurlburt, G., 2023. Can large language models better predict software vulnerability? IT Prof. 25 (3), 4–8.

Lewowski, T., Madeyski, L., 2022. How far are we from reproducible research on code smell detection? A systematic literature review. Inf. Softw. Technol. 144, 106783.

Li, X., Chang, X., Board, J.A., Trivedi, K.S., 2017. A novel approach for software vulnerability classification. In: Proceedings of the 64th IEEE/ACM International Conference on Reliability and Maintainability Symposium. RAMS, IEEE, pp. 1–7.

Li, J., Li, G., Li, Y., Jin, Z., 2023. Enabling programming thinking in large language models toward code generation. arXiv preprint arXiv:2305.06599.

Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., Chen, Z., 2021. Sysevr: A framework for using deep learning to detect software vulnerabilities. IEEE Trans. Dependable Secure Comput. 19 (4), 2244–2258.

Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., Zhong, Y., 2018. Vuldeepecker: A deep learning-based system for vulnerability detection. In: Proceedings of the 25th ACM Annual Network and Distributed System Security Symposium. NDSS, The Internet Society.

Lin, G., Wen, S., Han, Q.-L., Zhang, J., Xiang, Y., 2020a. Software vulnerability detection using deep neural networks: a survey. Proc. IEEE 108 (10), 1825–1848.

Lin, G., Xiao, W., Zhang, J., Xiang, Y., 2020b. Deep learning-based vulnerable function detection: A benchmark. In: Proceedings of the 21st ACM Information and Communications Security: International Conference. ICICS, Springer, pp. 219–232.

Lin, G., Zhang, J., Luo, W., Pan, L., De Vel, O., Montague, P., Xiang, Y., 2019. Software vulnerability discovery via learning multi-domain knowledge bases. IEEE Trans. Dependable Secure Comput. 18 (5), 2469–2485.

Lin, G., Zhang, J., Luo, W., Pan, L., Xiang, Y., De Vel, O., Montague, P., 2018. Cross-project transfer representation learning for vulnerable function discovery. IEEE Trans. Ind. Inform. 14 (7), 3289–3297.

Liu, X., Ji, K., Fu, Y., Tam, W., Du, Z., Yang, Z., Tang, J., 2022. P-tuning: Prompt tuning can be comparable to fine-tuning across scales and tasks. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics. ACL, Association for computational Linguistics, pp. 61–68.

Liu, X., Tan, Y., Xiao, Z., Zhuge, J., Zhou, R., 2023. Not the end of story: An evaluation of ChatGPT-driven vulnerability description mappings. In: Proceedings of the 61th Annual Meeting of the Association for Computational Linguistics. ACL, Association for Computational Linguistics, pp. 3724–3731.

Lu, G., Ju, X., Chen, X., Pei, W., Cai, Z., 2024. GRACE: Empowering LLM-based software vulnerability detection with graph structure and in-context learning. J. Syst. Softw. 212, 112–236.

Lu, J., Yu, L., Li, X., Yang, L., Zuo, C., 2023. Llama-reviewer: Advancing code review automation with large language models through parameter-efficient fine-tuning. In: 2023 IEEE 34th International Symposium on Software Reliability Engineering. ISSRE, IEEE, pp. 647–658.

Masana, M., Liu, X., Twardowski, B., Menta, M., Bagdanov, A.D., Van De Weijer, J., 2022. Class-incremental learning: survey and performance evaluation on image classification. IEEE Trans. Pattern Anal. Mach. Intell. 45 (5), 5513–5533.

Nachtigall, M., Schlichtig, M., Bodden, E., 2022. A large-scale study of usability criteria addressed by static analysis tools. In: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis. ISSTA, ACM, pp. 532–543.

Ozturk, O.S., Ekmekcioglu, E., Cetin, O., Arief, B., Hernandez-Castro, J., 2023. New tricks to old codes: Can AI chatbots replace static code analysis tools? In: Proceedings of the 7th ACM European Interdisciplinary Cybersecurity Conference. EICC, ACM, pp. 13–18.

Parisi, G.I., Kemker, R., Part, J.L., Kanan, C., Wermter, S., 2019. Continual lifelong learning with neural networks: A review. Neural Netw. 113, 54–71.

Pecorelli, F., Di Nucci, D., De Roover, C., De Lucia, A., 2019. On the role of data balancing for machine learning-based code smell detection. In: Proceedings of the 3rd ACM SIGSOFT International Workshop on Machine Learning Techniques for Software Quality Evaluation (MaLTeSQuE). ACM, pp. 19–24.

Pornprasit, C., Tantithamthavorn, C., 2024. Fine-tuning and prompt engineering for large language models-based code review automation. Inf. Softw. Technol. 107523.

Purba, M.D., Ghosh, A., Radford, B.J., Chu, B., 2023. Software vulnerability detection using large language models. In: Proceedings of the 34th IEEE International Symposium on Software Reliability Engineering Workshops. ISSREW, IEEE, pp. 112–119.

Shi, F., Chen, X., Misra, K., Scales, N., Dohan, D., Chi, E.H., Schärli, N., Zhou, D., 2023. Large language models can be easily distracted by irrelevant context. In: Proceedings of the 40th ACM International Conference on Machine Learning. ICML, PMLR, pp. 31210–31227.

Steenhoek, B., Rahman, M.M., Jiles, R., Le, W., 2023. An empirical study of deep learning models for vulnerability detection. In: Proceedings of the 45th IEEE/ACM International Conference on Software Engineering. ICSE, pp. 2237–2248.

Sun, Y., Wu, D., Xue, Y., Liu, H., Wang, H., Xu, Z., Xie, X., Liu, Y., 2024. Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. pp. 1–13.

Telang, R., Wattal, S., 2007. An empirical analysis of the impact of software vulnerability announcements on firm stock price. IEEE Trans. Softw. Eng. 33 (8), 544–557.

Thapa, C., Jang, S.I., Ahmed, M.E., Camtepe, S., Pieprzyk, J., Nepal, S., 2022. Transformer-based language models for software vulnerability detection. In: Proceedings of the 38th ACM Annual Computer Security Applications Conference. ACSAC, ACM, pp. 481–496.

Tomas, N., Li, J., Huang, H., 2019. An empirical study on culture, automation, measurement, and sharing of devsecops. In: 2019 International Conference on Cyber Security and Protection of Digital Services (Cyber Security). IEEE, pp. 1–8.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al., 2023. Llama: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971.

Tsipenyuk, K., Chess, B., McGraw, G., 2005. Seven pernicious kingdoms: A taxonomy of software security errors. IEEE Secur. Privacy 3 (6), 81–84.

Wei, Y., Sun, X., Bo, L., Cao, S., Xia, X., Li, B., 2021. A comprehensive study on security bug characteristics. J. Software Evolut. Process 33 (10), e2376.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q.V., Zhou, D., et al., 2022. Chain-of-thought prompting elicits reasoning in large language models. Adv. Neural Inf. Process. Syst. 35, 24824–24837.

White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., Schmidt, D.C., 2023. A prompt pattern catalog to enhance prompt engineering with ChatGPT. arXiv preprint arXiv:2302.11382.

Wu, Y., Jiang, N., Pham, H.V., Lutellier, T., Davis, J., Tan, L., Babkin, P., Shah, S., 2023. How effective are neural networks for fixing security vulnerabilities. In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis. pp. 1282–1294.

Zhang, Y., Li, Y., Cui, L., Cai, D., Liu, L., Fu, T., Huang, X., Zhao, E., Zhang, Y., Chen, Y., et al., 2023a. Siren's song in the AI ocean: A survey on hallucination in large language models. arXiv preprint arXiv:2309.01219.

Zhang, K., Li, Z., Li, J., Li, G., Jin, Z., 2023b. Self-edit: Fault-aware code editor for code generation. arXiv preprint arXiv:2305.04087.

Zhang, C., Liu, H., Zeng, J., Yang, K., Li, Y., Li, H., 2023c. Prompt-enhanced software vulnerability detection using ChatGPT. arXiv preprint arXiv:2308.12697.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., et al., 2024. Judging llm-as-a-judge with mt-bench and chatbot arena. Adv. Neural Inf. Process. Syst. 36.

Zhou, X., Jin, Y., Zhang, H., Li, S., Huang, X., 2016. A map of threats to validity of systematic literature reviews in software engineering. In: Proceedings of the 23rd IEEE International Conference on Asia-Pacific Software Engineering Conference. APSEC, IEEE, pp. 153–160.

Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y., 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. Adv. Neural Inf. Process. Syst. 32, 10197–10207.

Zhou, X., Zhang, T., Lo, D., 2024. Large language model for vulnerability detection: Emerging results and future directions. In: Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results. pp. 47–51.

Zou, D., Wang, S., Xu, S., Li, Z., Jin, H., 2019. μVulDeePecker: A deep learning-based system for multiclass vulnerability detection. IEEE Trans. Dependable Secure Comput. 18 (5), 2224–2236.

Yanjing Yang is a Ph.D. candidate in the State Key Laboratory for Novel Software Technology at the Nanjing University. His research mainly focuses on software security, AI security and LLM for security.

Xin Zhou is an assistant researcher at the Nanjing University. He was awarded Ph.D from Nanjing University. He undertakes research in software engineering, in particular software architecture, software security and quality, blockchain-oriented software engineering, empirical and evidence-based software engineering, and LLM4SE. He has published over 20 peer-reviewed papers in high quality international conferences and journals (e.g. ICSE/FSE/IST/JSS), and won a Best Paper Award from APSEC2016.

Runfeng Mao is a Ph.D. candidate in the State Key Laboratory for Novel Software Technology at the Nanjing University. His research mainly focuses on software security, DevSecOps.

Jinwei Xu is a Ph.D. candidate in the State Key Laboratory for Novel Software Technology at the Nanjing University. His research mainly focuses on software security, software supply chain.

Lanxin Yang is an assistant researcher at the Nanjing University. He was awarded Ph.D from Nanjing University. He has published over 10 peer-reviewed papers in high quality international conferences and journals (e.g. ICSE/FSE/IST).

Yu Zhang is a Master's candidate in the State Key Laboratory for Novel Software Technology at the Nanjing University. His research mainly focuses on software security.

Haifeng Shen is Professor of Information Technology within the Discipline of Engineering and Information Technology, Faculty of Science and Engineering, Southern Cross University. Previously, I had been continuously working in academia across multiple institutions including Nanyang Technological University, Flinders University, and Australian Catholic University. Throughout my career, I have played many leadership roles such as head of discipline of information technology, research lab director, HDR discipline chairperson, and director of teaching programs. I have also served on several important committees such as university academic board, Vice Chancellor's HDR strategy advisory group, and university human research ethics committee. He has published over 120 research papers at international conferences and journals, including

flagship and top venues such as ACM Transactions on Computer Human Interaction, International Journal of Human Computer Studies, ACM SIGCHI Conference on Human Factors in Computing Systems, ACM Conference on Computer Supported Cooperative Work and Social Computing, and IEEE/ACM International Conference on Automated Software Engineering.

He Zhang is a Full Professor of Software Engineering and the Director of DevOps+ Research Laboratory at the Nanjing University, China, also a Principal Scientist with CSIRO, Australia. He was awarded B.Eng (NUAA), M.PM (USyd) and Ph.D (UNSW). Prof. Zhang is an internationally recognized researcher and expert in software engineering. He joined academia after seven years working in software industry, developing software systems in the areas of aerospace and complex data management. He undertakes research in software engineering, in particular software process (modeling, simulation, analytics and improvement), software architecture, software security, blockchain-oriented software engineering, empirical and evidence-based software engineering, service-oriented computing, and data-driven software engineering. He has published over 150 peer-reviewed papers in high quality international conferences and journals, and won 11 Best/Distinguished Paper Awards from several prestigious international conferences and journals. He is a Member of the IEEE Computer Society and the ACM (SIGSOFT), a Senior Member of China Computer Federation (CCF), and serves on the Steering Committees and Program Committees of a number of high quality international conferences in software engineering community.